What Is Gen AI?

Gen AI is a type of Artificial Intelligence which creates a new content such as – text, images, videos, music and code – by understanding existing pattern, mimicking human creativity.

AI generally circulates around only 2 perspective

1. Builder perspective – The one who build complete LLM
2. User perspective – The one who use LLM API to perform actions, complete task or Agentic AI

What is LangChain?

LangChain is **open-source framework** that **helps in building LLM based application**. It provides **modular components, end-to-end tools** which helps to develop complex AI application such as chatbot, question-answer system, retrieval-augmented generation and autonomous agents.

- Its free, open source and actively developing
- Support all major LLMs
- Simplifies LLM based development using built in functions and properties
- Built in support for all major tools like DB connectivity, AWS or remote DB
- Support all major GenAI use cases

Benefits of LangChain

- **Concept of chain** – Output of one component will be given to input to another component automatically without writing manual steps. (Can perform series of tasks)(serial chains, parallel chains, conditional chains)
- **Model Agnostic Development** – easy to change LLM models without changing application configuration
- **Complete ecosystem** – Vast support for all types of components like have numerous databases, have multiple number of embedding models, support all type of doc reader, multiple text splitters and many more
- **Memory & state handling** – In conversation memory and state management to retrieve history

Components of LangChain

1. **Models** – Models are the **core interfaces through which application interact with AI model** (OpenAI, Anthropic, Gemini)
    a. Language Models – Text input, text output (LLM, ChatModel)
    b. Embedding Models – text input, vector output
2. **Prompts** – Inputs which are given to models
    a. Dynamic and reusable prompts
    b. Role based prompts
    c. Few shots prompt (examples-based prompts)
3. **Chains** – Perform multiple steps in single Query, passing **output of 1st step (1st component) to 2nd step (2nd component) as input** without any human interaction or without coding
4. **Indexes** – Indexes connect your application to external knowledge
    a. Document reader (PDF, Website, Database)

   b. Text Splitter – Split text in small chunks on basis of number of records, pages, paragraph
   c. Vector Store – Stores the vectors created by embedding text
   d. Retrieval – Used to retrieve data using search techniques from vector stores
5. **Memory** – Helps to memories the context of the previous questions or previous chat
   a. conversationBufferMemory – Stores and passes complete history conversation with new query
   b. conversationBufferWindowMemory – Stores and passes last n number of chats with new query
   c. summarizer-based memory – Stores summary of memory which will updated after every frequent interval and passes with new query
   d. custom memory – stores user preferences and facts and passes with new query
6. **Agents** – Agents are the **chatbot with reasoning capabilities and tools**. On the basis of reasoning capabilities agent determine what to do and how and use proper tool to complete that task

Usability of components:

Prompts:

1. **Static prompt** – Prompt which we get from user and directly send to LLM
2. **Dynamic Prompts** – We create template for prompt in which we pass some specific values so main motive of agent will preserve and user will not enter another irrelevant prompt (we use promptTemplate ())

Benefits of PromptTemplate over directly string appended/concat prompt

1. Default Validations
2. Can create reusable prompts
3. Supports chains

***PromptTemplate use will use in case of single short message mean follower questions can not be asked but of list of messages where follower questions are allowed, we use following method:

Messages: Used to structure conversations

1. SystemMessage – Message given by system (Assigned role, description of agent)
2. HumanMessage – Input given by user
3. AIMessage – Answer given by LLM

***For static human message we directly use above three messages in list format but for dynamic messages we need to use ChatPromptTemplate with above three messages same like PromptTemplate

Message Placeholder – When user came back **after a long time** or **created a new chat** and asked **question about previous conversations** then we need to provide **context(history) of that conversation** to answer correctly which is managed by message placeholder.
We fetch the history from DB and place it between system message and new query using message placeholder.

Structured Output

Structured output is practice of having language models return response in well-defined data format (e.g. JSON), rather than free form text.

Why we need structured output (use cases) ?

1. **Data extraction** – if we need to store some data in DB, then we can't directly add free form text in DB
2. **API building** –
3. **Agents** – In agent to provide payload to the tools we mostly use JSON structure

Three types of defined structured output:

1. **TypedDict** – We can specify the type of values but validation is not possible
2. **Pydantic** – We can specify type of the values, validation is also possible, can provide default values too, but cross language not supported
3. **JSON** – We can specify types of values, validation is also possible, majorly used in cross language application.

There are 2 modes in with_structure_output function (can pass as parameter)

a. **JSON mode** – Typical JSON output
b. **Function mode** – When we need to call the function passing the output as parameter then we use this method, mostly used in agents while calling tools

***There are some LLM which directly give structured output with minimal or single line of code using with_structure_output function but for the LLM which directly not gives structured output in that case we need to use output parse method

Output Parsers:

Helps to convert raw LLM response into structure format. Ensures consistency, validation and easy of use in application

a. **StrOutputParser** – Parse the output in plain text format
b. **JSONOutputParser** – Parse the output in JSON format. It does not enforce schema
c. **StructuredOutputParser** – Parse the output in JSON format based on predefined schema, it does not enforce validations.
d. **PydanticOutputParser** – Parse the output in pydantic object enforces both schema as well as validations

Runnables:

Runnables are unit of work, a standard building block that takes an input, does a specific task, and gives an output

Before Runnables, different parts of LangChain (like prompts, models, and parsers) used different commands and rules to run (format for prompt and predict for models). This made code messy and hard to connect

With Runnables, **every component works the exact same way.** Whether it is a prompt template, an AI model, or your own custom Python function, they all share core standard methods –

- **Invoke** – Run the block with a single input
- **Batch** – Run multiple inputs at the same time
- **Stream** – Get the output piece-by-piece as it is being generated (great for real-time text)

Two types of runnables –

1. **Task specific Runnables** – Core components of LangChain which are converted into runnables, they have their own purpose (e.g. ChatOpenAI, PromptTemplate, Retrievers)
2. **Primitive Runnables** – Special types of runnables which helps to connect different task specific runnables (e.g. – RunnableParallel, RunnableSequence, RunnableBranch, RunnablePassThrough, RunnableLambda)

RAG Components –

Document Loader:

Document loaders are the component used to load the data from different source of documents (txt, pdf, csv, web, DB) into a standardise format (a document object), which can used for chunking, embedding, retrieval and generation

Document object – Document(page_content="…….", metadata="……")

Different examples of loaders from lanchain_community.document_loaders

- **TextLoader** – Used to load data from .txt file
- **PyPDFLoader** – Used to load data from pdf, return the list of document objects where each object represents each page of pdf. It internally uses PyPDF library of python so it dependable also it not suitable with scanned pdf or complex layouts
- **DirectoyLoader** – Used to load all files in directory, we can load complete directory.
- **WebBaseLoader** – Used to extract the content from webpage. Not suitable for JavaScript heavy web pages also works well only with static pages
- **CsvLoader** – Use to load data from csv files

Text Splitters:

Text splitting is the process of breaking large chunks of text (like article, PDFs, HTML pages or books) into smaller, manageable pieces that can LLM handle effectively.

Why splitting is important

1. **Overcoming model limitation** – LLM model comes with token limit if we pass large amount of data with query there might be chance of hit the token limit very early, so generally prefer to pass relevant information only
2. **Downstream tasks** –

- **Embeddings** – It is observed that short chunk able to capture more meaning in vector rather than big chunks
- **Sematic search** – It is easy to search with smaller chunks and will bet getter result
- **Summarization** – Small chunks help LLM ko be on track; Larger chunks sometimes lead hallucinations

3. **Optimizing computational resources** – Smalls chunks require less process and well as less memory so help to optimize resources and allow parallel execution

Types of text splitters

1. **Length based text splitter** – Split the text on the basis of character count no matter sentence or word is breaking in between (e.g. 200 characters in each chunk) (CharacterTextSplitter)
2. **Text-Structure base splitter** – Based on structure of text it split text using separators like (para: \n\n, newline: \n, space), we can pass maximum size of chunk so in one chunk its not get oversized (RecursiveCharacterTextSplitter)
3. **Document structure-based Text splitting** – Worked same as Text-Structure base splitter just separators are different, in which we can split markdown document (using \n{1,6}, ```\n, \n***\n, \n---+\n like these separators) and also we can separate code files (using \nClass, \ndef, \n\tDef like these separators) (RecursiveCharacterTextSplitter.from_language)
4. **Semantic Meaning base splitting** – This splitter **generates vector of each sentence and compare each vector with its next corresponding vector and highest semantic sentences kept in one chunk** and where semantic meaning or score changes abruptly on that point it split the text in next chunk

Vector Stores:

A vector store is a system designed to store and retrieve data represented as numerical vector.

Key Features:

1. **Storage** – Ensure that vector and its related metadata are retained, weather in memory for quick operations or on a hard-disk for large scale use
2. **Similarity search** – Helps retrieve vector which is most similar to query vector
3. **Indexing** – Provide data structure or methods that enables fast similar search on high dimensional vectors
4. **CRUD operations** – Can perform add, read, update and delete operations

Vector Database:

Vector database is advance level of vector stores **with all properties of vector stores and with following additional properties**

- Distributed architecture
- Backup and restore feature
- ACID properties (Atomicity, Consistency, Isolation, Durability)
- Concurrency control (multiple operations at a same time)
- Authentication (access-based retriever)

Some common LangChain vector store – FAISS, Pinecone, **Chroma**, Qdrant, Waviate

***Every vector database is vector store but every vector store is not vector database

Retrievers:

Retrievers is a component of LangChain that fetches relevant document from a data source in response to the user query

Types of retrievers:

1. **Data source retrievers:**
    - **Wikipedia retriever** – Helps to fetch data from Wikipedia; It hits the Wikipedia Api and search for relevant content in Wikipedia
    - **Vector store retriever** – Helps to search and fetch document from vector stores based on semantic similarity using vector embeddings
2. **Search strategy retrievers:**
    - **Maximum marginal retriever** – Information retrieval algorithm **used to reduce redundancy** in the retrieved results while maintaining high relevance with query (It **compare resultant documents with each other's** too so same document with different sentence structure should not appear as redundant fetch)
    - **Multi query retriever** – Sometimes query may have ambiguity or single query might not capture all the ways information is phrased in document (It basically **pass the user query to LLM and ask LLM to remove ambiguity and divide into multiple queries** for same topics and based on these multiple queries it fetches documents)
    - **Contextual compression retriever** – Advance retriever that improves retrieval quality by **compressing documents after retrieval – keeping only relevant content** based on user's query (It first fetch the highest similarity documents with query, but now there is possibility that **document is having other data too which might not needed for this query**, so fetched document along with **query is given to LLM and LLM will trim the document and kept only required specific data** from document)

RAG (Retrieval-Augmented Generation)

Retrieval-Augmented Generation (RAG) is an artificial intelligence technique that improves a large language model's accuracy by fetching facts from an external knowledge base before writing an answer.

Why RAG is needed?

Suppose we want answer related to data which is private in nature like company policy, company databases or personal assistant which answer questions related to person then a LLM can't answer that questions because LLM does not trained on that data.
OR
Suppose we want answer related to things that are happing currently or happened few time ago then LLM can't answer that as each LLM has its own pre-training cutoff date e.g. If training of LLM is done on 31 June 2026 then incidents after 31 June 2026 was not in memory of LLM so LLM can't answer that
OR
Suppose we want facts but due to volume in LLM memory sometime LLM may hallucinate so to avoid this

we can provide external data source answer ask LLM to draw facts only from that source instead of using that wide memory

Option to RAG –
Instead of RAG we can go with Fine-tuning LLM (pre-training LLM again on our data) by having model but having a model and run on our server can have so much computational cost and for fine tuning it required a data with 10k+ records

Steps of RAG

1. Indexing
2. Retrieval
3. Augmentation
4. Generation

- **Indexing –**
    a. **Document Loader** – Load documents from external source like DB, Web, PDF
    b. **Chunking (Text splitting)** – Convert large document into small chunks for getting semantic search and to pass the only relevant context in available context limit
    c. **Embedding generation** – Convert each chuck into dense vector which capture its semantic meaning using embedding model
    d. **Stores in Vector Store** – We store original text and its corresponding vectors into vector store like FAISS, Chroma for local and Pinecone, Qdrant, Weaviate on cloud
- **Retrieval –**
  We retrieve, find the most relevant piece of information from pre-build index based on user query, we embedded the user query and find sematic search and retrieval methods to find out relevant piece.
- **Augmentation –**
  It is process of creating prompt with the help of retrieved context and query. We can assign some role, validation to the prompt like “If this is not mention in context then just day – I Don’t know”, so LLM will combinedly get its role, what to do, context and query
- **Generation –**
  Generation is final stage where we pass the retrieval and augmented context to LLM for generate response and LLM gives responses for that query

There are some improvements methods within above steps to efficiency of our RAG application

1. **UI based enhancement (good look and feel for agent)**
2. **Evaluation (Test efficiency)**
    a. Ragas – Library to evaluate faithfulness, relevance, context precision, context recall
    b. LangSmith – Used to trace/generate logs
3. **Indexing**
    a. Document Ingestion (fix error in document – pre-processing)
    b. Text splitting (semantic based splitter)
    c. Vector Store (advanced vector store)

4. **Retrieval**
    a. Pre-Retrieval
        i. Query rewriting using LLM
        ii. Multi query generation
        iii. Domain aware routing (multiple retrievers for different types of queries in same app)
    b. During retrieval
        i. MMR
        ii. Hybrid retrieval
        iii. Reranking (reranking fetched documents using LLM)
    c. Post Retrieval
        i. Contextual compression
5. **Augmentation**
    a. Prompt templating
    b. Answer grounding (ask LLM to not go beyond context, or avoid hallucinations)
    c. Context window optimization (trim context to meaningful context for context limitation)
6. **Generation**
    a. Answer with citation
    b. Guard railing
7. **System design**
    a. Multimodal
    b. Agentic (RAG with internal tool access to perform some tasks)
    c. Memory based

Tools

A tools is just a Python function (or API) that is packaged in a way the LLM can understand and Call when needed.

LLM are great at Reasoning, Language generation, But They can't do things like Access live data, Do reliable math, Call APIs, Run code, Interact with a DB so to perform this step we need tools.

An AI agent is an LLM-powered system that can autonomously think, decide and take actions using tools or APIs to achieve goal.

Ways to create Tools

1. **Using @tool decorator** (Most Common)
2. **Using StructuredTool & Pydantic** – A Structured Tool in LangChain is a special type of tool where the input to the tool follows a structured schema, typically defined using a Pydentic model
3. **Using BaseTool class** – BaseTool is the abstract class for all tools in LangChain, It defines the core structure and interface that any tool must follow, whether it's simple one-liner or full customized function, All other tool type like @tool, StructureTool are built on top of BaseTool